

Earl James Williams A. Aliñgasa
Mr. Mark Phil B. Pacot
CSC 112 - BNO1
9 August 2025

GRADED PRE-ACTIVITY

1 Regular Expression Task

The regular expression given to us is

$$RE = 1^*00^*1(0 + 1)^*.$$

Each component with the operators $*$ and $+$ can be substituted for substrings within the set defined by said operators:

- 1^* : $\{\epsilon, "1", "11", \dots\}$;
- 0^* : $\{\epsilon, "0", "00", \dots\}$;
- $0 + 1$: $\{"0", "1"\}$; hence
- $(0 + 1)^*$: $\{\epsilon, "0", "1", "00", "01", "10", "11", "000", \dots\}$.

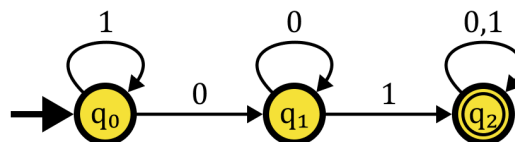
For our example string, we will make the following substitutions:

- 1^* for ϵ ,
- 0^* for "0", and
- $(0 + 1)^*$ for "10".

And so our example string becomes

$$\begin{aligned}\text{Example String} &= 1^*00^*1(0 + 1)^* \\ &= (\epsilon)0(0)1(10) \\ &= 00110.\end{aligned}$$

One possible DFA that can accept this regular expression is given below:



Formally:

DFA = $(Q, \Sigma, \delta, q_0, F)$ such that

$$Q = \{q_0, q_1, q_2\};$$

$$\Sigma = \{0, 1\};$$

$$q_0 = q_0;$$

$$F = \{q_2\}; \text{ and}$$

δ is the transition mapping function defined by below.

	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$* q_2$	q_2	q_2

We can prove that our example string, "00110" is accepted by the proposed DFA by walking through the transitions:

1. $\delta(q_0, 0) = q_1$;
2. $\hat{\delta}(q_0, "00") = \delta(\delta(q_0, 0), 0) = \delta(q_1, 0) = q_1$;
3. $\hat{\delta}(q_0, "001") = \delta(\hat{\delta}(q_0, "00"), 1) = \delta(q_1, 1) = q_2$;
3. $\hat{\delta}(q_0, "0011") = \delta(\hat{\delta}(q_0, "001"), 1) = \delta(q_2, 1) = q_2$; and finally
3. $\hat{\delta}(q_0, "00110") = \delta(\hat{\delta}(q_0, "0011"), 0) = \delta(q_2, 0) = q_2$;

The final state q_2 was reached from the starting state q_0 , hence the example string "00110" is indeed accepted by the DFA that adheres to the regular expression $1^*00^*1(0 + 1)^*$.

2 Machine Code Translation Task

The instructions that must be created using Assembly are as follows:

```
x: int
x = 100
```

One possible program on Windows would look like this:

```
section .text
    global start
    extern ExitProcess    ; This is needed for proper exiting.

section .bss
    x        resd 1        ; Memory has to be allocated for the integer x.
                        ; 4 bytes are reserved in this case to align with C.
                        ; This was an arbitrary choice.

start:
    mov     dword [x], 100 ; The value 100 is stored within x.
    push    0              ; This status call indicates no errors, like C does.
    call    ExitProcess    ; The program has to be exited properly.
```